

Primero aplicaría un **QA de party_id + data_date** antes de construir la distribución por estado. No haría todavía el join final hasta confirmar que una misma fecha no tiene estados distintos.

1. Validar la granularidad de bc3.padd

```
padd_party_date_qa <- sdf_sql(  
  sc,  
  "  
  SELECT  
    party_id,  
    data_date,  
  
    COUNT(*) AS rows_per_party_date,  
    COUNT(DISTINCT state_code) AS distinct_states  
  
  FROM bc3.padd  
  
  GROUP BY  
    party_id,  
    data_date  
  
  HAVING COUNT(*) > 1  
  
  ORDER BY  
    distinct_states DESC,  
    rows_per_party_date DESC  
  
  LIMIT 100  
  "  
) %>%  
  collect()
```

padd_party_date_qa

Aquí quiero ver especialmente **distinct_states**.

Si obtenemos, por ejemplo:

rows_per_party_date	distinct_states
3	1
2	1
5	1

no hay un problema conceptual: existen filas repetidas en el snapshot, pero todas indican el mismo estado.

Si aparece:

```
rows_per_party_date  distinct_states
2                    2
```

entonces necesitamos una regla adicional porque el mismo party tiene dos estados diferentes en el mismo `data_date`.

También ejecutaría un resumen:

```
padd_party_date_summary <- sdf_sql(
  sc,
  "
  WITH x AS (

    SELECT
      party_id,
      data_date,
      COUNT(*) AS rows_per_party_date,
      COUNT(DISTINCT state_code) AS distinct_states

    FROM bc3.padd

    GROUP BY
      party_id,
      data_date
  )

  SELECT
    COUNT(*) AS party_date_combinations,

    SUM(
      CASE WHEN rows_per_party_date > 1
        THEN 1 ELSE 0
      END
    ) AS duplicated_party_dates,

    SUM(
      CASE WHEN distinct_states > 1
        THEN 1 ELSE 0
      END
    ) AS party_dates_multiple_states,

    MAX(rows_per_party_date)
      AS max_rows_same_party_date,

    MAX(distinct_states)
      AS max_states_same_party_date

  FROM x
```

```
"
) %>%
  collect()

padd_party_date_summary
```

La métrica crítica es:

party_dates_multiple_states

Idealmente:

0

2. Si **party_dates_multiple_states = 0**

Entonces podemos crear una versión limpia de **padd** eliminando duplicados exactos conceptualmente:

```
sdf_sql(
  sc,
  "
  CREATE OR REPLACE TEMP VIEW padd_state_clean AS

  SELECT DISTINCT
    party_id,
    data_date,
    state_code

  FROM bc3.padd
  "
)
```

Ahora sí hacemos el join point-in-time.

3. Seleccionar el último estado conocido antes de **observation_date**

Como para este análisis quieres estudiar los positivos:

```
positive_state_point_in_time <- sdf_sql(
```

sc,
"

WITH positives AS (

SELECT

party_id,
observation_date,
observation_year,
label_3m

FROM bu_frd.model_features_ab_final_2022_2025

WHERE label_3m = 1

),

candidate_states AS (

SELECT

f.party_id,
f.observation_date,
f.observation_year,
f.label_3m,

p.data_date AS state_data_date,
p.state_code,

ROW_NUMBER() OVER (

PARTITION BY

f.party_id,
f.observation_date

ORDER BY

p.data_date DESC

) AS rn

FROM positives f

LEFT JOIN padd_state_clean p

ON f.party_id = p.party_id

AND p.data_date <= f.observation_date

)

SELECT

party_id,
observation_date,

```
observation_year,  
label_3m,
```

```
state_data_date,  
state_code
```

```
FROM candidate_states
```

```
WHERE rn = 1  
"
```

```
) %>%  
collect()
```

```
positive_state_point_in_time
```

La lógica es:

$$\left[\boxed{\max(\text{data_date})} \text{ such that } \text{data_date} \leq \text{observation_date} \right]$$

por cada party.

Eso es exactamente lo que queremos.

4. QA después del join

Antes de calcular la distribución por estados debemos confirmar que terminamos con:

62,599

parties positivos, una fila cada uno.

```
positive_state_qa <- sdf_sql(  
  sc,  
  "
```

```
WITH positives AS (
```

```
  SELECT  
    party_id,  
    observation_date
```

```
  FROM bu_frd.model_features_ab_final_2022_2025
```

```
  WHERE label_3m = 1  
)
```

candidate_states AS (

SELECT

f.party_id,
f.observation_date,
p.data_date,
p.state_code,

ROW_NUMBER() OVER (

PARTITION BY

f.party_id,
f.observation_date

ORDER BY

p.data_date DESC

) AS rn

FROM positives f

LEFT JOIN padd_state_clean p

ON f.party_id = p.party_id

AND p.data_date <= f.observation_date

)

SELECT

COUNT(*) AS total_rows,

COUNT(DISTINCT party_id)
AS distinct_parties,

SUM(

CASE WHEN state_code IS NULL
THEN 1 ELSE 0

END

) AS parties_without_state

FROM candidate_states

WHERE rn = 1

"

) %>%

collect()

positive_state_qa

Esperamos:

```
total_rows      62599
distinct_parties 62599
```

`parties_without_state` puede ser mayor que cero; eso no necesariamente es un problema, pero hay que reportarlo.

5. Distribución final de positivos por estado

Una vez pase el QA:

```
positive_state_distribution <- sdf_sql(
  sc,
  "
  WITH positives AS (

    SELECT
      party_id,
      observation_date

    FROM bu_frd.model_features_ab_final_2022_2025

    WHERE label_3m = 1
  ),

  candidate_states AS (

    SELECT
      f.party_id,
      f.observation_date,

      p.data_date,
      p.state_code,

      ROW_NUMBER() OVER (

        PARTITION BY
          f.party_id,
          f.observation_date

        ORDER BY
          p.data_date DESC
```

```

    ) AS rn

FROM positives f

LEFT JOIN padd_state_clean p

    ON f.party_id = p.party_id
    AND p.data_date <= f.observation_date
),

final_state AS (

    SELECT
        party_id,
        observation_date,
        state_code

    FROM candidate_states

    WHERE rn = 1
)

SELECT
    COALESCE(state_code, 'MISSING')
        AS state_code,

    COUNT(*) AS positive_parties,

    COUNT(*) * 1.0 /
    SUM(COUNT(*)) OVER ()
        AS pct_of_positives

FROM final_state

GROUP BY
    COALESCE(state_code, 'MISSING')

ORDER BY
    positive_parties DESC
"
) %>%
collect()

positive_state_distribution

```

Primero ejecuta **los dos QA del paso 1**. Si `party_dates_multiple_states = 0`, puedes continuar directamente con `padd_state_clean` y el point-in-time join. Si aparecen

múltiples `state_code` distintos para el mismo `party_id + data_date`, ahí conviene detenernos y definir el desempate antes de seguir.